21.3. Describe the steps associated with user experience testing for an app.

21.4. What is the objective of security testing? Who performs this testing activity?

21.5. Assume that you are developing a MobileApp to access an online pharmacy (`YourCornerPharmacy.com`) that caters to senior citizens. The pharmacy provides typical functions but also maintains a database for each customer so that it can provide drug information and warn of potential drug interactions. Discuss any special usability or accessibility tests for this MobileApp.

21.6. Assume that you have implemented a Web service that provides a drug interaction–checking function for `YourCornerPharmacy.com` (see Problem 21.5). Discuss the types of component-level tests that would have to be conducted on the mobile device to ensure that the MobileApp accesses this function properly.

21.7. Is it possible to test every configuration that a MobileApp is likely to encounter in the production environment? If it is not, how do you select a meaningful set of configuration tests?

21.8. Describe a security test that might need to be conducted for the YourCornerPharmacy MobileApp (Problem 21.5). Who should perform this test?

21.9. What is the difference between testing that is associated with interface mechanisms and testing that addresses interface semantics?

21.10. What is the difference between testing for navigation syntax and navigation semantics?

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

22 SOFTWARE CONFIGURATION MANAGEMENT

Change is inevitable when computer software is built and can lead to confusion when you and other members of a software team are working on a project. Confusion arises when changes are not analyzed before they are made, recorded before they are implemented, reported to those with a need to know, or controlled in a manner that will improve quality and reduce error. Babich [Bab86] suggests an approach that will minimize confusion, improve productivity, and reduce the number of mistakes when he writes: "Configuration management is the art of identifying, organizing, and controlling modifications to the software being built by a programming team. The goal is to maximize productivity by minimizing mistakes."

KEY CONCEPTS

baselines 441
change control 448
change management, mobility and agile . . . 453
change management process 447
configuration audit 452
configuration management, elements of . . . 440
configuration objects 441
content management 455
continuous integration 446
identification 452
integration and publishing 455
repository. 453
SCM process 448
software configuration items 438
status reporting. 452
version control. 445



QUICK LOOK

What is it? When you build computer software, change happens. And because it happens, you need to manage it effectively. Software configuration management (SCM), also called change management, is a set of activities designed to manage change.

Who does it? Everyone involved in the software process is involved with change management to some extent, but specialized support positions are sometimes created to manage the SCM process.

Why is it important? If you don't control change, it controls you. And that's never good. It's very easy for a stream of uncontrolled changes to turn a well-run software project into chaos. As a consequence, software quality suffers and delivery is delayed.

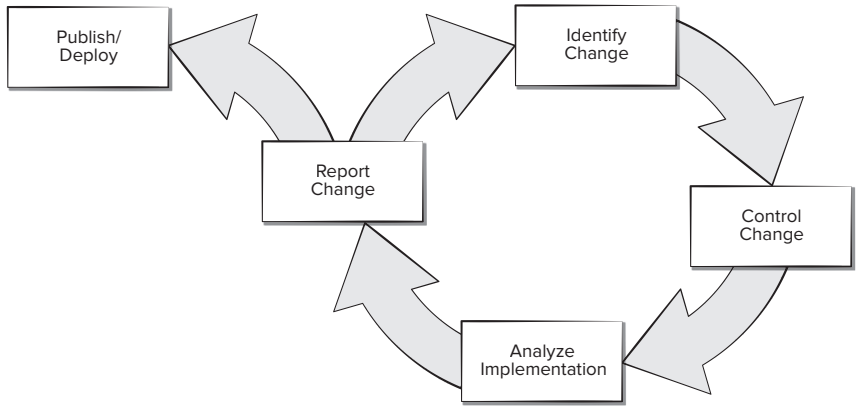
What are the steps? Because many work products are produced when software is built, each must be uniquely identified. Once this is accomplished, mechanisms for version and change control can be established.

What is the work product? A software configuration management plan defines the project strategy for change management. Changes result in updated software products that must be retested and documented, without breaking the project schedule or the production versions of the software products.

How do I ensure that I've done it right? When every work product can be accounted for, traced, controlled, tracked, and analyzed; when everyone who needs to know about a change has been informed—you've done it right.

FIGURE 22.1

Software configuration management work flow



Software configuration management (SCM) is an umbrella activity that is applied throughout the software process. Typical SCM work flow is shown in Figure 22.1. Because change can occur at any time, SCM activities are developed to (1) identify change, (2) control change, (3) ensure that change is being properly implemented, and (4) report changes to others who may have an interest.

It is important to make a clear distinction between software support and software configuration management. Support (Chapter 27) is a set of software engineering activities that occur after software has been delivered to the customer and put into operation. Software configuration management is a set of tracking and control activities that are initiated when a software engineering project begins and terminates only when the software is taken out of operation.

A primary goal of software engineering is to improve the ease with which changes can be accommodated and reduce the amount of effort expended when changes must be made. In this chapter, we discuss the specific activities that enable you to manage change.

22.1 SOFTWARE CONFIGURATION MANAGEMENT

The output of the software process is information that may be divided into three broad categories: (1) computer programs (both source level and executable forms), (2) work products that describe the computer programs (targeted at various stakeholders), and (3) data or content (contained within the program or external to it). In Web design or game development, managing changes to the multimedia content items can be more demanding than managing the changes to the software or documentation. The items that comprise all information produced as part of the software process are collectively called a *software configuration*.

As software engineering work progresses, a hierarchy of *software configuration items* (SCIs)—a named element of information that can be as small as a single UML diagram or as large as the complete design document—is created. If each SCI simply led to other SCIs, little confusion would result. Unfortunately, another variable enters the process—*change*. Change may occur at any time, for any reason. In fact, the *first law of system*

engineering [Ber80] states: "No matter where you are in the system life cycle, the system will change, and the desire to change it will persist throughout the life cycle."

What is the origin of these changes? The answer to this question is as varied as the changes themselves. However, there are four fundamental sources of change:

- New business or market conditions dictate changes in product requirements or business rules.
- New stakeholder needs demand modification of data produced by information systems, functionality delivered by products, or services delivered by a computer-based system.
- Reorganization or business growth or downsizing causes changes in project priorities or software engineering team structure.
- Budgetary or scheduling constraints cause a redefinition of the system or product.

Software configuration management is a set of activities that have been developed to manage change throughout the life cycle of computer software. SCM can be viewed as a software quality assurance activity that is applied throughout the software process. In the sections that follow, we describe major SCM tasks and important concepts that help us to manage change.

22.1.1 An SCM Scenario

This section is extracted from [Dar01].[1]

> A typical configuration management (CM) operational scenario involves several stakeholders: a project manager who is in charge of a software group, a configuration manager who is in charge of the CM procedures and policies, the software engineers who are responsible for developing and maintaining the software product, and the customer who uses the product. In the scenario, assume that the product is a small one involving about 15,000 lines of code being developed by an agile team with four developers. (Note that other scenarios of smaller or larger teams are possible, but, in essence, there are generic issues that each of these projects face concerning CM.)

At the operational level, the scenario involves various roles and tasks. For the project manager or team leader, the goal is to ensure that the product is developed within a certain time frame. Hence, the manager monitors the progress of development and recognizes and reacts to problems. This is done by generating and analyzing reports about the status of the software system and by performing reviews on the system.

The goals of the configuration manager (who on a small team may be the project manager) are to ensure that procedures and policies for creating, changing, and testing of code are followed, as well as to make information about the project accessible. To implement techniques for maintaining control over code changes, this manager introduces mechanisms for making official requests for changes, for evaluating proposed changes with the development team, and ensuring the changes are acceptable to the product owner. Also, the manager collects statistics about components in the software system, such as information determining which components in the system are problematic.

---

1 Special permission to reproduce "Spectrum of Functionality in CM Systems" by Susan Dart [Dar01], © 2001 by Carnegie Mellon University is granted by the Software Engineering Institute.

For the software engineers, the goal is to work effectively. There must be a mechanism to ensure that simultaneous changes to the same component are properly tracked, managed, and executed. This means engineers do not unnecessarily interfere with each other in the creation and testing of code and in the production of supporting work products. But, at the same time, they try to communicate and coordinate efficiently. Specifically, engineers use tools that help build a consistent software product. They communicate and coordinate by notifying one another about tasks required and tasks completed. Changes are propagated across each other's work by merging files. Mechanisms exist to ensure that, for components that undergo simultaneous changes, there is some way of resolving conflicts and merging changes. A history is kept of the evolution of all components of the system along with a log with reasons for changes and a record of what actually changed. The engineers have their own workspace for creating, changing, testing, and integrating code. At a certain point, the code is made into a baseline from which further development continues and from which variants for other target machines are made.

The customer uses the product. Because the product is under CM control, the customer follows formal procedures for requesting changes and for indicating bugs in the product.

Ideally, a CM system used in this scenario should support all these roles and tasks; that is, the roles determine the functionality required of a CM system. The project manager sees CM as an auditing mechanism; the configuration manager sees it as a controlling, tracking, and policy-making mechanism; the software engineer sees it as a changing, building, and access control mechanism; and the customer sees it as a quality assurance mechanism.

22.1.2 Elements of a Configuration Management System

In her comprehensive white paper on software configuration management, Susan Dart [Dar01] identifies four important elements that should exist when a configuration management system is developed:

- **Component elements.** A set of tools coupled within a file management system (e.g., a database) that enables access to and management of each software configuration item.
- **Process elements.** A collection of procedures and tasks that define an effective approach to change management (and related activities) for all constituencies involved in the management, engineering, and use of computer software.
- **Construction elements.** A set of tools that automate the construction of software by ensuring that the proper set of validated components (i.e., the correct version) have been assembled.
- **Human elements.** A set of tools and process features (encompassing other CM elements) used by the software team to implement effective SCM.

These elements (to be discussed in more detail in later sections) are not mutually exclusive. For example, component elements work in conjunction with construction elements as the software process evolves. Process elements guide many human activities that are related to SCM and might therefore be considered human elements as well.

22.1.3 Baselines

Change is a fact of life in software development. Customers want to modify requirements. Developers want to modify the technical approach. Managers want to modify the project strategy. Why all this modification? The answer is really quite simple. As time passes, all constituencies know more (about what they need, which approach would be best, and how to get it done and still make money). Most software changes are justified, so there's no point in complaining about them. Rather, be certain that you have mechanisms in place to handle them.

A *baseline* is a software configuration management concept that helps you to control change without seriously impeding justifiable change. The IEEE [IEE17] defines a baseline as:

> A specification or product that has been formally reviewed and agreed upon, that thereafter serves as the basis for further development, and that can be changed only through formal change control procedures.

Before a software configuration item becomes a baseline, change may be made quickly and informally. However, once a baseline is established, changes can be made, but a specific, formal procedure must be applied to evaluate and verify each change.

In the context of software engineering, a baseline is a milestone in the development of software. A baseline is marked by the delivery of one or more software configuration items that have been approved as a consequence of a technical review (Chapter 16). For example, the elements of a design model have been documented and reviewed. Errors are found and corrected. Once all parts of the model have been reviewed, corrected, and then approved, the design model becomes a baseline. Further changes to the program architecture (documented in the design model) can be made only after each has been evaluated and approved. Although baselines can be defined at any level of detail, the most common software baselines are shown in Figure 22.2.

The progression of events that lead to a baseline is also illustrated in Figure 22.2. Software engineering tasks produce one or more SCIs. After SCIs are reviewed and approved, they are placed in a *project database* (also called a *project library* or *software repository* and discussed in Section 22.5). Be sure that the project database is maintained in a centralized, controlled location. When a member of a software engineering team wants to make a modification to a baselined SCI, it is copied from the project database into the engineer's private workspace. However, this extracted SCI can be modified only if SCM controls (discussed later in this chapter) are followed. The arrows in Figure 22.2 illustrate the modification path for a baselined SCI.

22.1.4 Software Configuration Items

We have already defined a software configuration item as information that is created as part of the software engineering process. In the extreme, an SCI could be considered to be a single section of a large specification or one test case in a large suite of tests. More realistically, an SCI is all or part of a work product (e.g., a document, an entire suite of test cases, a named program component, a multimedia content asset, or a software tool).

In reality, SCIs are organized to form configuration objects that may be cataloged in the project database with a single name. A *configuration object* has a